dewlock

• SYSTEM ARCHITECTURE · DIAGRAM-FIRST

# System architecture.

Intent-firewall copilot for agentic Sui DeFi. A keyless agent builds one unsigned PTB, a deterministic Guardian re-derives the math and fail-closes, and your wallet signs the literal artifact you reviewed.

**14**
GUARDIAN GATES

**0**
USER-FUND KEYS SERVER-SIDE

**3**
TRUST ZONES, ONE BOUNDARY

**1137**
TESTS GREEN

Sui mainnet · live
dewlock.vercel.app

apps/web + @dewlock/{agent, sui, walrus}

• TRUST ZONES & SYSTEM CONTEXT

# Three trust zones, one boundary keys never cross.

The agent and Guardian run server-side but stay **keyless** — they only ever emit an *unsigned* bundle. Signing happens in your wallet; settlement, valuation and receipts happen on Sui and Walrus.

### Client, keyless server, and on-chain — left to right

context · 01


Client — holds keys | Server — keyless / deterministic | On-chain — Sui + Walrus + DeFi

Trust boundary (no keys cross) | Unsigned build / dry-run

1. you state intent in natural language → the keyless agent
2. the agent returns ONE unsigned PTB → your wallet
3. the Guardian dry-runs it; builders read/write protocol state on Sui
4. your wallet signs → the tx + a Walrus receipt land on-chain

• STACK

Sui · DeepBook v3 · Cetus · Aftermath · NAVI · Suilend · SuiNS · Wormhole · Walrus · Upstash Redis · memwal · Seal · Mastra · Next.js 16 · React 19 · TypeScript · Tailwind v4 · Vercel AI Gateway · Claude · Gemini · Move

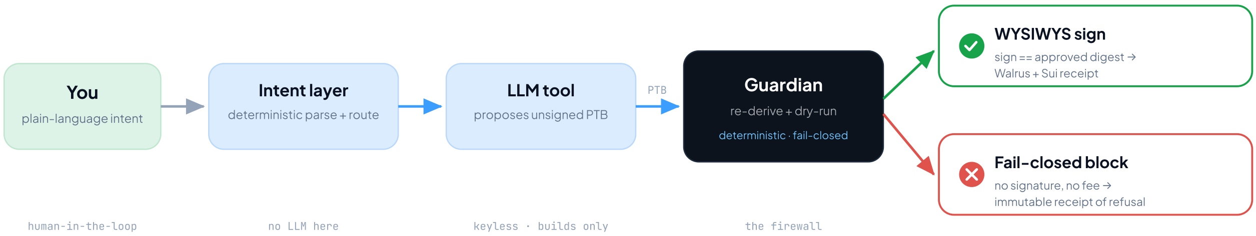
• THE FIREWALL PIPELINE

# The agent builds, the Guardian verifies, you sign.

A plain-language intent compiles into one unsigned PTB. Before any signature, a deterministic Guardian re-derives the math and dry-runs the exact bundle — every value move runs the full gauntlet.

## Intent → sign — the firewall pipeline

flow · 02




LLM proposes, deterministic Guardian re-derives and fail-closes — any single gate failing blocks the whole PTB.

## The 14 Guardian gates — every PTB runs the gauntlet

gates · 03

| | GATE | WHAT IT RE-DERIVES — ONE LINE EACH |
|---|---|---|
| ● | **value & integrity** | |
| 1 | `server caps` | per-tx + per-day USD caps; invalid cap config blocks everything; borrow has its own inflow cap. |
| 2 | `trusted price` | real oracle value (SUI/USD live, USDC=$1); no price → block. LST outflow uses a floor formula. |
| 3 | `min-out re-derive` | recompute swap min-output from on-chain decimals + same route; runs for every swap (anti-sandwich). |
| 4 | `authoritative value` | re-value from dry-run net deltas; block when outflow > 1.5× declared (outflow_mismatch). |
| 5 | `WYSIWYS digest` | approvedDigest = sha256(kindBytes); signed bytes must equal the reviewed kind. |
| ● | **identity & correctness** | |
| 6 | `allowlist` | every MoveCall {pkg::mod::fn} must be pre-approved; runs first so unknown targets never process. |
| 7 | `action-shape` | PTB call-set must match exactly one actionType template; one value action per PTB. |
| 8 | `coin-type provenance` | coinTypeIn/out verified on-chain via CoinMetadata; identity by TYPE, never display symbol. |
| 9 | `injection provenance` | per-field argProvenance; a derived recipient triggers a confirm gate (anti prompt-injection). |
| 10 | `SuiNS lookalike` | homoglyph-normalized edit-distance vs verified contacts (catches 888-l.sui-style spoofs). |
| 11 | `staking constraints` | LST coin-type vs curated map; provider-keyed shapes (afSUI ≠ haSUI cannot cross). |
| 12 | `post-tx health factor` | NAVI borrow/withdraw; devInspect HF < threshold (1.6) blocks; full-withdraw cannot leave debt. |
| ● | **dependency fail-closed** | |
| 13 | `dry-run + effects` | dry-run exact PTB bytes incl. gas; RPC/dry-run failure or unknown target blocks, never proceeds. |
| 14 | `composite recipe` | atomic single-sign: closed registry · target multiset · coin-type linkage · delta/owner + dual caps. |

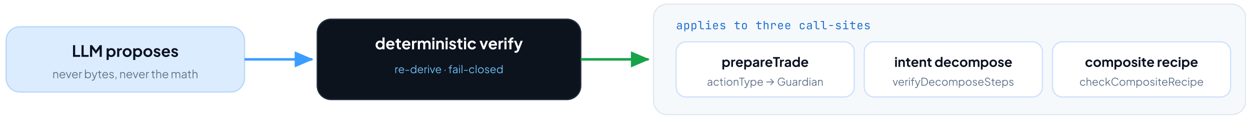
SPECIAL ALGORITHMS

# One moat pattern, three places it shows up.

The LLM only ever **proposes**; deterministic code **verifies** and fail-closes. The same pattern guards trade preparation, multi-intent decomposition, and atomic composite recipes.

## The moat — LLM proposes, deterministic code verifies

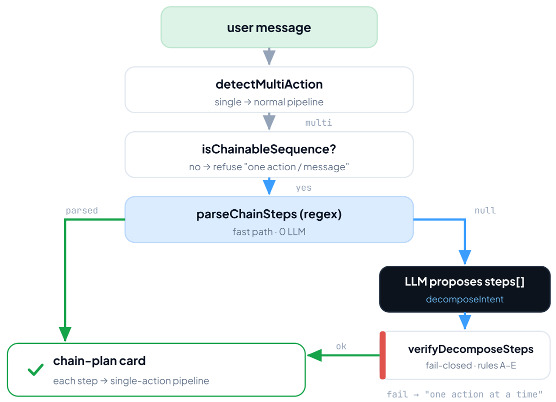
pattern · 04




Same router that guards a trade also guards a decomposition step and a composite leg — proposal and verification never share authority.

## Hybrid multi-intent decompose

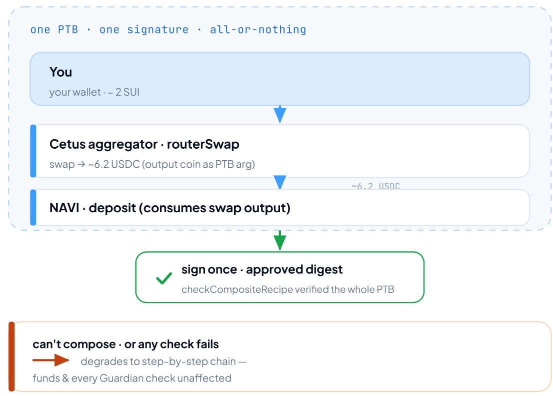
flow · 05




verifyDecomposeSteps rejects all on: **A** <2 steps · **B** category ∉ {swap,lend,stake,send} · **C** router disagrees · **D** step 0 not explicit amount · **E** a step hides a 2nd action.

## Atomic composite — one signature

flow · 06




recipe `swap_lend_v1`: the swap-output coin feeds the deposit *inside* the PTB — a Move abort in either leg reverts both.